

Network Calls

Basics

Every **server** works essentially the same way:

- A **client** makes a **request**
- The server sends back a **response**

In our case, the **client is the iOS app** instead of a web browser

HTTP

The hypertext transfer protocol (**HTTP**) is typically used for this **Request-Response cycle**. There are several Request Methods but the four we will talk about are:

- **GET** - retrieve a resource
- **POST** - send data
- **PUT** - send data (typically used to *update*)
- **DELETE** - delete a resource

HTTP Status Codes

- **2xx** - Success
 - 200 - OK
- **4xx** - Client Error
 - 400 - BadRequest, 404 - Not Found
- **5xx** - Server Error
 - 500 - Internal Server Error

RESTful API Example

GET a movie by an id:

- `http://{baseUrl}/v1/movie/{id}`

POST (create) a movie with id

- `http://{baseUrl}/v1/movie/{id}`

The difference is that the POST request will have a JSON body that will contain the movie

RESTful API Example

GET an array of action movies with year equal to 1972:

- `http://{baseUrl}/v1/movies?year=1972&genre=action`

REST Calls in iOS

Example Service Call

```
// Set up url
let baseUrl = URL(string: "http://default-environment.jpkaumtzk.us-east-2.elasticbeanstalk.com/v1")!

let namespace = "4220-EXAMPLE"

let getMoviesUrl = baseUrl.appendingPathComponent("movies?namespace=\(namespace)")

// Set up request
var request = URLRequest(url: getMoviesUrl)
request.httpMethod = "GET"

// Configure the URLSession task
let task = URLSession.shared.dataTask(with: request) { (data, response, error) in
    // Write our code to handle the response from the server
}

// Start the network call task
task.resume()
```

URL

- Identifies the **location** of a resource, for example:
 - An item on a **remote server**

```
import Foundation

// The URL construction can fail, so make sure it's
// valid or else handle the optional properly
var baseUrl = URL(string: "http://www.google.com")!

baseUrl.appendPathComponent("search")
baseUrl.appendPathComponent("sharks")

// URL manages the forward slashes for you
// when you work with path components
print(baseUrl.absoluteString) // 'http://www.google.com/search/sharks'
```

URLRequest

- An object that defines the type of request to send
- Common items you need to set:
 - **URL** - where to send the request
 - **HTTP Method** - GET, POST, etc
 - **HTTP Body** - for POST, PUT, and others methods with a body
 - **HTTP Headers** - for authentication, locale, content-type..
 - **Timeout** - how long to wait before terminating the request

URLSession

- Coordinates network data transfer tasks
- Natively supports the data, file, ftp, http, and https URL schemes
- Performs a `URLSessionTask`
- The tasks within a given `URLSession` share a common *session configuration object*
 - We will use the `URLSession.shared` singleton instance

URLSessionDataTask

- The type of `URLSessionTask` that we will use in our examples
- Takes in a `URLRequest` and a `completionHandler` **closure** which will execute when the task finishes
- `completionHandler` will be called on a **background thread**
- `completionHandler` has three parameters:
 - `Error?` - An optional error object
 - `Data?` - The data returned by the server (if successful)
 - `URLResponse?` - Metadata about the response
- Starts off in a suspended state
- Invoke `resume()` method to start it

Data

- Literally, just a **collection of bytes**
- In the context of network calls, this is the payload returned from the server
- Used all over the place in iOS

URLResponse

- Contains ***metadata*** associated with the server's response:
 - **StatusCode**: 200, 404, etc.
 - **HTTP** Response Headers
 - **Expected content length**: size of the data in bytes
- **Notes**:
 - Does ***NOT*** contain the data actually returned from the call
 - **MUST** be cast as an HTTPURLResponse to get more meaningful data from it (above attributes, etc)

Error

- protocol
- Represents an **error value**
- Can be thrown
- In the context of network calls, this could include:
 - A status code **NOT** in the 200s
 - A timeout
 - Decoding issue
 - etc...
- Also used all over the place in iOS

Result<Success, Failure>

- A generic enum representing either a success or a failure
- Includes an associated value for each case
- Failure case associated value **MUST** be a subclass of `Error`
- Useful for parsing service call responses
- Example:

```
typealias ServiceCallResult = Result<Data, ServiceCallError>
```